COS 201: PROGRAMMING I

Mid-Semester Lab Assessment Report

Done by Odama Miracle Akwanufo

With Registration Number: 2025/B/CSC/0742

Student ID:301867975

Project: Student Course Management System (Java)

1. What the System Does

The Student Course Management System is a Java console program that helps a student record and manage the courses they are taking for the semester. From a numbered menu the user can add courses (code, title and credit units), view all courses in a table, search for a course by its code, calculate the total credit units, and save or load the course list from a text file (courses.txt). The program is split into four files: Course.java (the data class), CourseManager.java (all the logic), InvalidCourseException.java (a custom exception) and Main.java (menu and user interface).

2. How Each Major Feature Works

Feature	How it works
1. Add Course	Main asks for code, title and units and passes the raw text to CourseManager.addCourse(). The code is trimmed and converted to upper case, the title is trimmed and put in Title Case, and the units are converted from text to an integer. Each value is validated before a new Course object is stored in an ArrayList.
2. View All Courses	A for loop goes through the ArrayList and prints each course in an aligned table with a serial number. If the list is empty, a friendly message is shown instead.
3. Search by Code	The user types a code (any casing). A recursive method compares it with each stored code and returns the position of the match, or -1 if none exists. The matching course details are then displayed.
4. Total Units	A recursive method adds the units of every course and the result is displayed together with the number of courses.
5. Save to File	Each course is written as one line in the format CODE|TITLE|UNITS using BufferedWriter inside a try-with-resources block.
6. Load from File	BufferedReader reads the file line by line in a while loop. Each line is split on '|' and passed through the same addCourse() validation, so corrupted lines are skipped and counted instead of crashing the program.
7. Exit	Choosing 7 ends the recursive menu (its base case) and the program stops.

3. Challenges Encountered and How Errors Were Handled

The main challenge was making the program stay stable when the user enters unexpected input. Errors were handled in these ways:

- **Custom exception:** InvalidCourseException is thrown for an invalid course code (must be 3 letters + 3 digits, e.g. COS201), an empty title, non-numeric units, units outside 1 to 6, or a duplicate course code. Main catches it and prints a clear error message, then returns to the menu.
- **NumberFormatException:** caught when the menu choice or the units are not numbers (e.g. typing "hello" or "abc"), so the program does not crash.
- **File exceptions:** FileNotFoundException is caught if the user loads before any file has been saved, and IOException covers other read/write problems. try-with-resources closes the files automatically, and the old list is restored if loading fails part-way.
- **Messy text:** string processing (trim(), toUpperCase(), regular-expression matching and Title Case conversion) means " cos 201 " and "use of   english" are cleaned up to COS201 and "Use of English".

- **Other checks:** saving an empty list is refused, searching with an empty code is rejected, and a menu choice outside 1 to 7 shows a message.

4. Where Recursion Was Applied

- **Recurring menu (Main.runMenu):** after every action the method calls itself to show the menu again. Its base case is choice 7 (Exit), which returns and unwinds the calls.
- **Recursive search (CourseManager.findIndexByCode):** checks the course at an index and, if it is not a match, calls itself with index + 1. Base cases: a match is found, or the end of the list is reached (return -1).
- **Recursive total (CourseManager.totalUnits):** returns the units at the current index plus totalUnits(index + 1). The base case returns 0 when the end of the list is reached.

5. Other Requirements Covered

Classes and objects (Course, CourseManager, custom exception); methods for every operation; a switch statement for menu decisions; loops (for, while); exceptions; and file reading/writing. Test runs are shown in the three screenshot files submitted with this report. To run: compile with javac *.java, then start with java Main.